UNIVERSIDAD AUTÓNOMA DE OCCIDENTE

Facultad de Ingeniería

Programa de Ingeniería Informática

PROYECTO FINAL

Plataforma de Biblioteca Universitaria

Seis estructuras de datos, base de datos con transacciones y despliegue web

Asignatura: Estructuras de Datos y Algoritmos II

Listas · Pilas · Colas · Árboles · Tries · Heaps · Grafos

Presentado por:

Salvador Rodriguez Velasco

Estudiante de sexto semestre

Presentado a:

Prof. Ing. Jonathan López Londoño

Santiago de Cali, octubre de 2026

Tabla de contenido

1. Introducción y objetivo 4
2. Alcance del sistema 4
2.1 Funcionalidades 4
2.2 Reglas de negocio 5
2.3 Fuera del alcance 5
3. Tecnologías utilizadas 5
4. Arquitectura 6
5. Base de datos y transacciones 6
5.1 Modelo de datos 6
5.2 Transacciones 7
6. Estructuras de datos: qué son, dónde se usan y por qué 8
6.1 Pila — carrito de devoluciones 8
6.2 Cola enlazada — lista de espera 9
6.3 Trie — buscador con autocompletado 9
6.4 Árbol N-ario — clasificación temática 10
6.5 Heap — vencimientos y ranking 10
6.6 Grafo ponderado — recomendaciones 10
7. Desarrollo paso a paso 11
Paso 1. Crear el repositorio con frontend, backend y código compartido 11
Paso 2. Definir los datos de ejemplo 11
Paso 3. Construir el backend con SQLite y transacciones 11
Paso 4. Implementar las seis estructuras 11
Paso 5. Construir las estructuras desde la base de datos 12
Paso 6. Repositorio con modo API y modo local 12
Paso 7. Interfaz 12
Paso 8. Pruebas automáticas 20
Paso 9. Control de versiones con ramas 21
8. Cómo ejecutar el programa 21
8.1 Requisitos 21
8.2 Ejecución completa (backend + frontend) 21
8.3 Solo frontend (modo local) 22
8.4 Pruebas y compilación 22
8.5 Solución de problemas 22

9. Publicación en servidores públicos 22
9.1 Backend en Render 22
9.2 Frontend en Netlify 23
9.3 Enlaces en el README 23
10. Detalle de requisitos 23
11. Conclusiones 24

1. Introducción y objetivo

Este documento presenta el **Proyecto Final** de la asignatura Estructuras de Datos y Algoritmos II: una **plataforma web para la biblioteca universitaria** de la Universidad Autónoma de Occidente. Explica el alcance del sistema, las tecnologías usadas, las estructuras de datos elegidas y el porqué de cada una, el desarrollo paso a paso y la manera de ejecutarlo y publicarlo.

Objetivo: integrar en un producto funcional las estructuras vistas en el curso (listas, pilas, colas, árboles, tries, heaps y grafos), cada una resolviendo un problema real de una biblioteca, con almacenamiento en base de datos y operaciones transaccionales.

La plataforma toma como base los ejercicios de la asignatura: la lista enlazada y doblemente enlazada), las pilas y colas, los árboles binarios y N-arios con DFS y BFS y los grafos con lista de adyacencia y react-d3-graph.

2. Alcance del sistema

2.1 Funcionalidades

Módulo	Qué permite hacer	Estructura principal
Inicio	Tablero con indicadores, préstamos por vencer, carrito de devoluciones, listas de espera y libros más prestados.	Heap, Pila, Cola
Catálogo	Buscar por título o autor con autocompletado, filtrar por facultad, área o materia, y agregar libros.	Trie, Árbol N-ario
Ficha del libro	Ver disponibilidad, prestar, reservar (lista de espera), registrar devoluciones y ver recomendaciones.	Cola, Grafo
Préstamos	Registrar préstamos con reglas y ver los activos ordenados por vencimiento.	Heap
Devoluciones	Reubicar el libro de la cima del carrito; si tiene reservas, se presta al primero de la cola.	Pila + Cola
Reservas	Ver y administrar las listas de espera de los libros agotados.	Cola
Recomendaciones	Grafo interactivo de libros relacionados, vecinos por peso y camino más corto entre dos libros.	Grafo
Usuarios	Registrar usuarios y ver sus préstamos activos, vencidos y reservas.	Base de datos
Estructuras	Tabla de las seis estructuras con sus números en vivo, explorador del Trie y estado de la base de datos.	Todas

2.2 Reglas de negocio

- Un usuario puede tener como máximo 5 préstamos activos.
- Un usuario con préstamos vencidos no puede hacer nuevos préstamos.
- No se puede prestar un libro que el usuario ya tiene, ni uno sin ejemplares disponibles.
- Solo se reservan libros agotados, y un usuario no puede repetir turno en la misma lista de espera.
- Un libro devuelto no vuelve de inmediato a la estantería: queda en el carrito hasta que el bibliotecario lo reubica. Si tiene reservas, pasa directo al primero de la cola.

2.3 Fuera del alcance

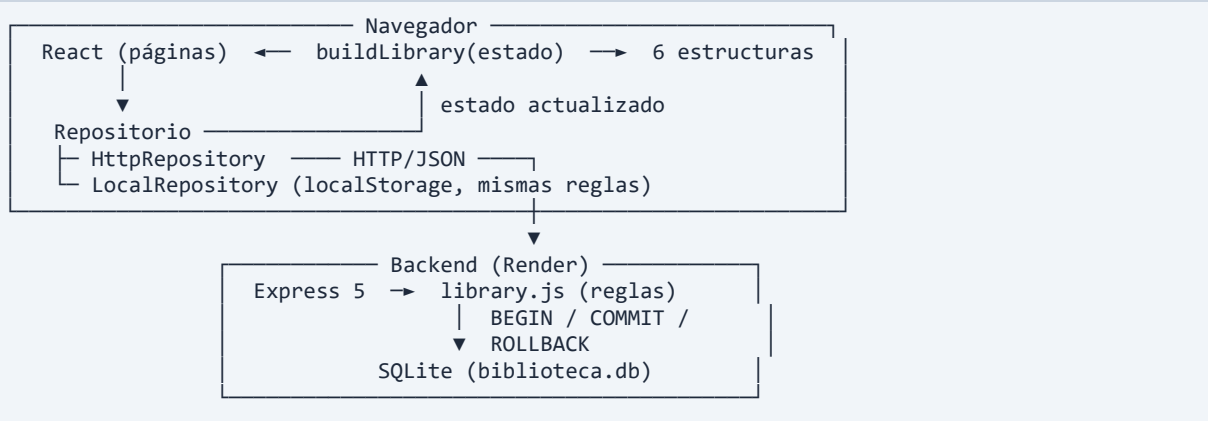
No se incluyen inicio de sesión con contraseñas, multas económicas, notificaciones por correo ni integración con el sistema académico. La plataforma está pensada para el bibliotecario, que opera en nombre de los usuarios.

3. Tecnologías utilizadas

Capa	Tecnología	Uso
Frontend	React 19 + Vite 8	Interfaz web de una sola página (SPA)
Frontend	React Router 7	Navegación entre los módulos
Frontend	react-d3-graph 2.6 + d3 5	Dibujo interactivo del grafo de recomendaciones (visto en la Clase 10)
Backend	Node.js 22 + Express 5	API REST
Base de datos	SQLite (módulo nativo node:sqlite)	Almacenamiento y transacciones sin instalar drivers
Pruebas	node:test	17 pruebas automáticas (9 del backend y 8 de las estructuras)
Control de versiones	Git	Una rama por funcionalidad, mezcladas en main
Despliegue	Netlify y Render	Frontend estático y backend Node.js

Las seis estructuras de datos están implementadas a mano en JavaScript, sin librerías. La única librería externa relacionada con estructuras es react-d3-graph, y solo se usa para dibujar el grafo; el grafo en sí es la clase `Graph` del proyecto.

4. Arquitectura



1. La interfaz pide el estado completo de la biblioteca (`GET /api/state`).
2. `buildLibrary` arma en memoria la pila, las colas, el Trie, el árbol, los heaps y el grafo a partir de las tablas.
3. Las páginas consultan esas estructuras: buscar, ordenar por vencimiento, recomendar, etc.
4. Cada acción (prestar, devolver, reubicar, reservar…) se envía al backend, que la ejecuta como transacción y responde con el estado actualizado; las estructuras se reconstruyen.
5. Si el backend no responde o no está configurado, el repositorio local aplica las mismas reglas (`shared/rules.js`) sobre una copia del estado y solo la confirma si todo sale bien. Así la plataforma publicada nunca queda sin funcionar.

5. Base de datos y transacciones

5.1 Modelo de datos

Tabla	Campos principales	Relación con las estructuras
categories	id, name, parent_id, code	Filas padre-hijo que forman el Árbol N-ario
books	id, code, title, author, year, publisher, category_id, total_copies, available_copies	Nodos del Grafo; palabras del Trie
users	id, code, name, program, role	Lectores
loans	id, book_id, user_id, loan_date, due_date, return_date	Préstamos activos en el Min-Heap; co-préstamos del Grafo
reservations	id, book_id, user_id, created_at, status	Cola de espera por libro (orden de created_at)
return_stack	id, loan_id, book_id, pushed_at, shelved_at	Pila del carrito (orden de pushed_at)

La base de datos protege la consistencia con restricciones: `CHECK (available_copies >= 0 AND available_copies <= total_copies)`, claves foráneas, códigos únicos y roles válidos.

5.2 Transacciones

Toda operación que modifica más de una tabla pasa por la función `transaction`: si cualquier validación falla a mitad de camino se ejecuta **ROLLBACK** y la base de datos queda exactamente como estaba.

```
export function transaction(db, fn) {
  db.exec('BEGIN IMMEDIATE');
  try {
    const result = fn();
    db.exec('COMMIT');
    return result;
  } catch (error) {
    db.exec('ROLLBACK');
    throw error;
  }
}
```

Operación	Pasos dentro de la misma transacción
Préstamo	Validar reglas → insertar en loans → available_copies − 1
Devolución	Marcar return_date → insertar en return_stack (push a la pila)
Reubicar	Tomar la cima de return_stack (pop) → si hay reserva en espera: marcarla atendida (dequeue) e insertar el préstamo para ese usuario; si no: available_copies + 1
Reserva	Validar que el libro esté agotado y que no haya duplicado → insertar en reservations (enqueue)
Cancelar reserva	Validar que esté en espera → status = cancelled
Nuevo libro / usuario	Validar datos y unicidad → insertar

La operación Reubicar es la que mejor muestra la combinación de estructuras y transacciones:

```
shelveTop() {
  return transaction(db, () => {
    // Cima de la pila = el último que entró (LIFO)
    const top = one(db, `SELECT * FROM return_stack WHERE shelved_at IS NULL
                         ORDER BY pushed_at DESC, id DESC LIMIT 1`);
    if (!top) throw new LibraryError(409, 'No hay libros en el carrito de devoluciones.');
    db.prepare('UPDATE return_stack SET shelved_at = ? WHERE id = ?').run(now, top.id);

    // Frente de la cola = la reserva más antigua (FIFO)
    const first = one(db, `SELECT * FROM reservations WHERE book_id = ? AND status = 'waiting'
                           ORDER BY created_at ASC, id ASC LIMIT 1`, top.book_id);
    if (first) {
      db.prepare("UPDATE reservations SET status = 'fulfilled' WHERE id = ?").run(first.id);
      insertLoan.run(top.book_id, first.user_id, now, dueDate);    // se le presta
      return { bookId: top.book_id, assignedTo: first.user_id };
    }
    db.prepare('UPDATE books SET available_copies = available_copies + 1 WHERE id =
?').run(top.book_id);
    return { bookId: top.book_id, assignedTo: null };
  });
}
```

6. Estructuras de datos: qué son, dónde se usan y por qué

La plataforma implementa seis estructuras: el Heap se agregó al Trie y al Árbol N-ario porque resuelve de forma natural los vencimientos y el ranking.

Rúbrica	Estructura	Problema que resuelve	Operación clave	Complejidad
Lineal	Pila	Carrito de devoluciones	push / pop	O(1)
Lineal	Cola (enlazada)	Lista de espera por libro	enqueue / dequeue	O(1)
Árbol	Trie	Buscador con autocompletado	buscar prefijo	O(m + k)
Árbol	Árbol N-ario	Clasificación temática	subárbol / suma recursiva	O(n)
Árbol	Heap	Vencimientos y ranking	peek / pop	O(1) / O(log n)
Grafo	Grafo ponderado	Recomendaciones	vecinos / BFS	O(grado) / O(V+E)

6.1 Pila — carrito de devoluciones

Estructura LIFO (Last In, First Out): el último elemento que entra es el primero que sale.

Se usa, cuando un usuario devuelve un libro, el ejemplar se pone encima del carrito del bibliotecario (push). Al reubicar, se toma siempre el de arriba (pop).

Es exactamente cómo funciona un carrito o una pila física de libros: no se saca el de abajo sin mover los de encima. Además, push, pop y peek son O(1).

```
push(value) { this.items.push(value); return this.items.length; }
pop()       { return this.items.length > 0 ? this.items.pop() : null; }
peek()      { return this.items.length > 0 ? this.items[this.items.length - 1] : null; }
```

6.2 Cola enlazada — lista de espera

Estructura FIFO (First In, First Out). Se implementó con nodos enlazados (`head`, `tail` y `next`,) en lugar de un arreglo con `shift()`, para que dequeue sea O(1) en vez de O(n).

Se usa: cada libro agotado tiene su cola de reservas. Reservar es enqueue; al reubicar un ejemplar devuelto se hace dequeue y se le presta al primero.

Garantiza que se atienda primero a quien reservó primero, que es la regla justa en una biblioteca. `positionOf` muestra a cada usuario su turno.

```
enqueue(value) {
  const node = new QueueNode(value);
  if (this.tail) this.tail.next = node; else this.head = node;
  this.tail = node;
  this.length++;
}
dequeue() {
  if (!this.head) return null;
  const node = this.head;
  this.head = node.next;            // solo se mueve el puntero: O(1)
  if (!this.head) this.tail = null;
  this.length--;
  return node.value;
}
```

6.3 Trie — buscador con autocompletado

El árbol de prefijos. Cada nodo es un carácter con un mapa de hijos; las palabras que comparten prefijo comparten el camino desde la raíz.

Se usa: se usan dos Tries. El primero indexa cada palabra de títulos y autores y apunta a los libros. Con él se busca: cada palabra escrita se trata como prefijo y se intersectan los resultados, así «clean mar» encuentra Clean Code y Clean Architecture de Robert C. Martin. El segundo guarda frases completas para el autocompletado mientras se escribe.

Encuentra todas las palabras que empiezan por un prefijo cuesta O(m) para llegar al nodo, más el recorrido de su subárbol (k resultados), sin revisar todo el catálogo. Se normaliza el texto (minúsculas y sin tildes), así que «algebra» encuentra «Álgebra».

```
insert(key, value) {
  let node = this.root;
  for (const ch of normalize(key)) {
    if (!node.children.has(ch)) node.children.set(ch, new TrieNode());
    node = node.children.get(ch);
  }
  if (!node.values.includes(value)) node.values.push(value);
}
// collect(prefijo): baja hasta el nodo del prefijo y recorre su subárbol (DFS)
```

6.4 Árbol N-ario — clasificación temática

Cada nodo tiene una lista de hijos de cualquier tamaño; se recorre con DFS (recursivo) y BFS (con cola).

Se usa: las 17 categorías forman el árbol Biblioteca UAO → facultades → áreas → materias. Se construye desde las filas (`id, parent_id`) de la base de datos con `ArbolNario.desdeFilas`. El panel de categorías es un componente recursivo.

La clasificación de una biblioteca es jerárquica. Al filtrar «Informática» deben aparecer los libros de todas sus materias: `subarbol(nodo)` las entrega en O(n), y el número de libros de cada categoría es una suma recursiva sobre su subárbol (`sumar`).

```
sumar(valorDe, nodo = this.raiz) {
  if (!nodo) return 0;
  return valorDe(nodo.valor) + nodo.hijos.reduce((s, h) => s + this.sumar(valorDe, h), 0);
}
```

6.5 Heap — vencimientos y ranking

El árbol binario completo guardado en un arreglo (hijos de i en 2i+1 y 2i+2) en el que cada padre tiene más prioridad que sus hijos; la raíz es siempre el más prioritario. La clase recibe un comparador, así sirve como min-heap o como max-heap.

Se usa: un min-heap de préstamos activos por fecha de vencimiento: la raíz es el préstamo más urgente o vencido, y el tablero muestra los cinco primeros. Un max-heap de libros por número de préstamos da el ranking de los más prestados.

Siempre se necesita el elemento más urgente, no la lista completa ordenada. `peek` es O(1) y `push` y `pop` son O(log n). `top(k)` extrae los k primeros de una copia sin modificar el heap.

```
push(value) {                 // agrega al final y SUBE mientras gane a su padre
  this.data.push(value);
  this.#siftUp(this.data.length - 1);
}
pop() {                       // saca la raíz, pone el último arriba y lo BAJA
  const top = this.data[0];
  const last = this.data.pop();
  if (this.data.length > 0) { this.data[0] = last; this.#siftDown(0); }
  return top;
}
```

6.6 Grafo ponderado — recomendaciones

Los vértices y aristas guardados con lista de adyacencia. Se conservan los métodos de la clase (`addNode`, `addEdge`, `searchNode`, `printAdjacency` y `printGraph`). Cada arista guarda además un peso y sus motivos, y si se agrega dos veces el peso se acumula.

Dónde se usa: cada libro es un nodo. Dos libros se unen si los prestó el mismo usuario (+3), si son del mismo autor (+2) o de la misma materia (+1). Las recomendaciones de un libro son sus vecinos ordenados

por peso. Con BFS se muestra el vecindario a uno o dos saltos y se calcula el camino más corto entre dos libros. El grafo se dibuja con react-d3-graph.

Por qué: las relaciones entre libros forman una red, no una jerarquía ni una secuencia. El grafo permite combinar varios criterios en un solo peso y explorar conexiones indirectas («Cien años de soledad → Refactoring → Clean Code»).

```
addEdge(id1, id2, weight = 1, reason = null) {
  for (const [a, b] of [[id1, id2], [id2, id1]]) {        // no dirigido
    const edge = this.adjList[a][b] ?? { weight: 0, reasons: [] };
    edge.weight += weight;                                 // el peso se acumula
    if (reason && !edge.reasons.includes(reason)) edge.reasons.push(reason);
    this.adjList[a][b] = edge;
  }
}
```

7. Desarrollo paso a paso

Paso 1. Crear el repositorio con frontend, backend y código compartido

```
mkdir biblioteca-uao && cd biblioteca-uao && git init
mkdir backend frontend shared docs
npm create vite@latest frontend -- --template react
cd frontend && npm install react-router-dom react-d3-graph d3@5
cd ../backend && npm init -y && npm install express cors
```

Como `react-d3-graph` declara compatibilidad solo con React 16, el frontend incluye un archivo `.npmrc` con `legacy-peer-deps=true` para que `npm install` funcione junto a React 19. Se comprobó en el navegador que el grafo se dibuja y responde a los clics.

Paso 2. Definir los datos de ejemplo

`shared/seed.js` crea 17 categorías, 31 libros, 8 usuarios, préstamos activos (tres de ellos vencidos), historial de préstamos, 7 reservas en 4 colas y 3 libros en el carrito. Las fechas se calculan desde el momento actual, así siempre hay préstamos vigentes y vencidos para probar.

Paso 3. Construir el backend con SQLite y transacciones

`backend/src/db.js` crea el esquema y carga los datos de ejemplo si la base está vacía. `library.js` implementa cada operación dentro de `transaction` y `app.js` expone la API REST. Los errores de negocio responden con su código HTTP (400, 404 o 409) y un mensaje en español.

Paso 4. Implementar las seis estructuras

En `frontend/src/structures/` se implementaron `Stack.js`, `Queue.js`, `Trie.js`, `ArbolNario.js`, `Heap.js` y `Graph.js`, cada una con comentarios que explican sus operaciones y su complejidad (sección 6).

Paso 5. Construir las estructuras desde la base de datos

`frontend/src/data/buildLibrary.js` recibe el estado (las tablas) y arma las estructuras: la pila con las devoluciones pendientes ordenadas por `pushed_at`, una cola por libro con las reservas ordenadas por `created_at`, los Tries con títulos y autores, el árbol desde `parent_id`, los heaps con los préstamos y conteos, y el grafo con los tres tipos de relación.

Paso 6. Repositorio con modo API y modo local

`services/repository.js` define dos implementaciones con los mismos métodos. `connectRepository` intenta conectarse al backend (`VITE_API_URL`); si no responde, usa el modo local, que aplica `shared/rules.js` dentro de `transaction` sobre una copia del estado y lo guarda en `localStorage`. La barra superior indica el modo activo: «API · SQLite» o «Modo local».

Paso 7. Interfaz

Cada panel tiene una etiqueta de color que indica qué estructura de datos lo alimenta. Estas son las pantallas principales:



Figura 1. Inicio: buscador, indicadores, préstamos por vencer (Heap) y carrito de devoluciones (Pila).

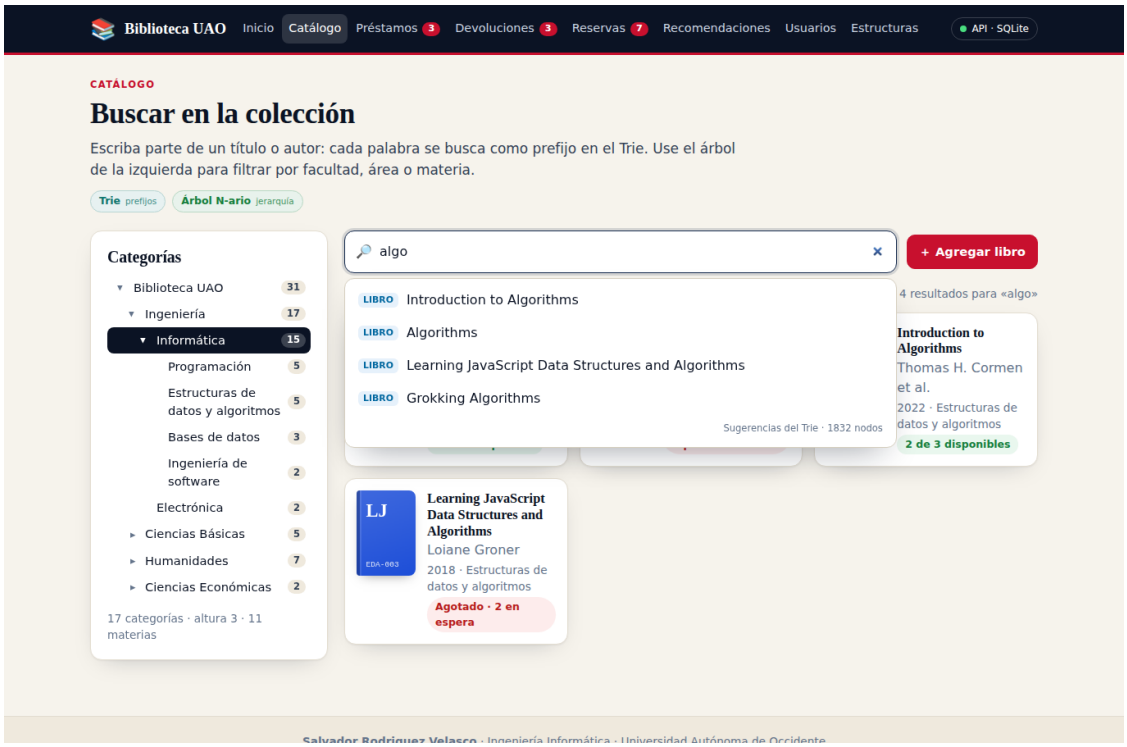


Figura 2. Catálogo: árbol de categorías (Árbol N-ario) y autocompletado del Trie para «algo» dentro de Informática.

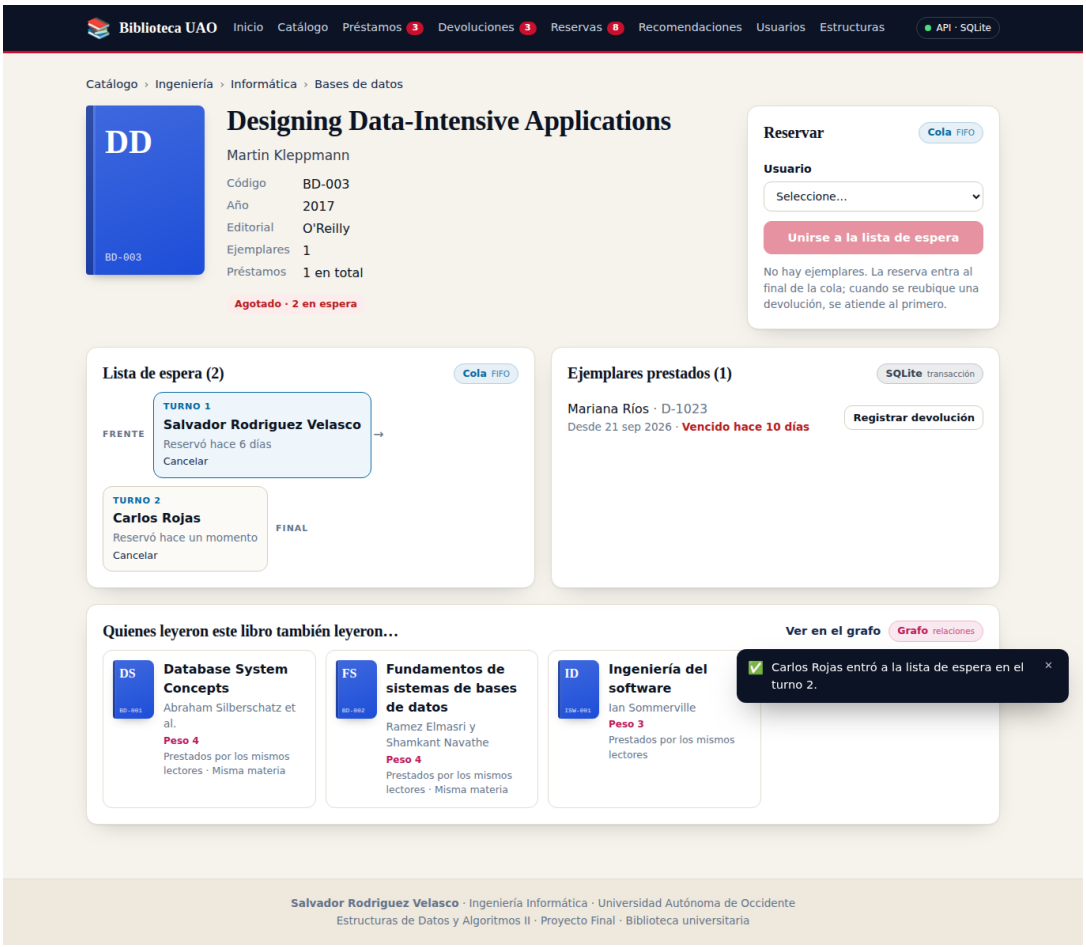


Figura 3. Ficha del libro agotado: lista de espera (Cola) con turnos, ejemplar prestado y recomendaciones (Grafo).

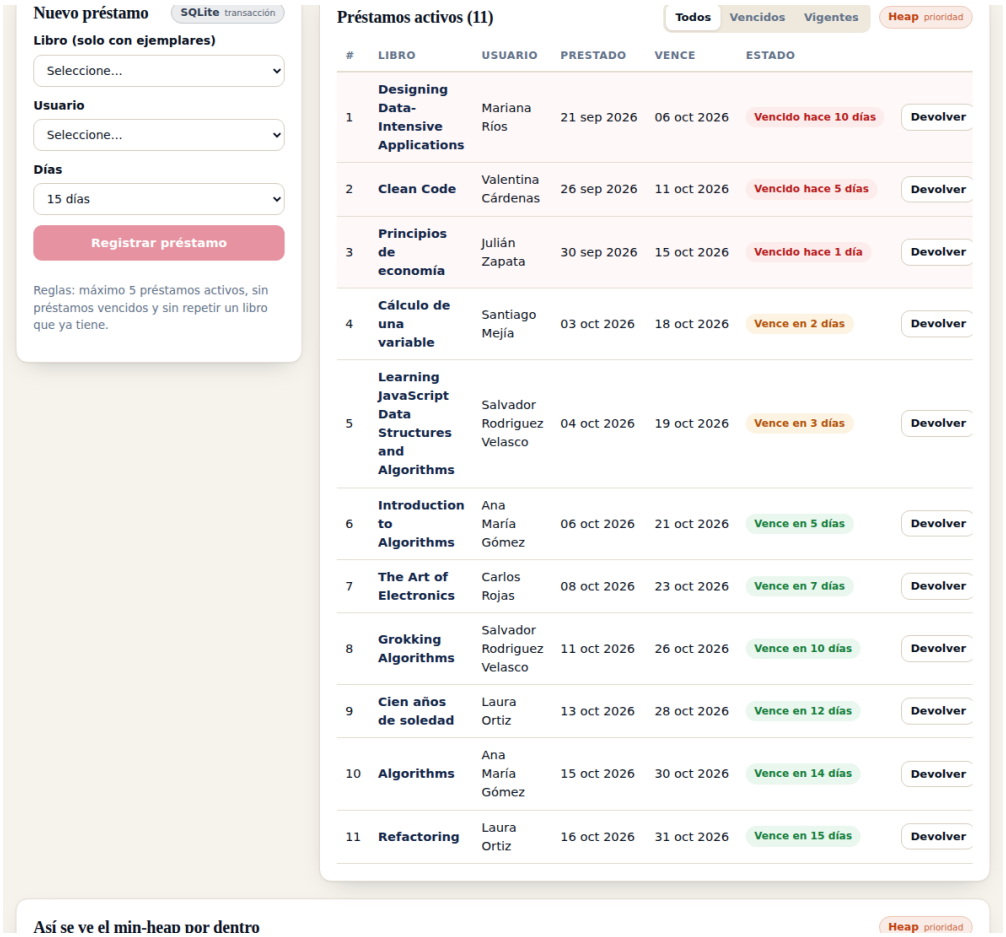


Figura 4. Préstamos activos extraídos del min-heap: primero los vencidos.

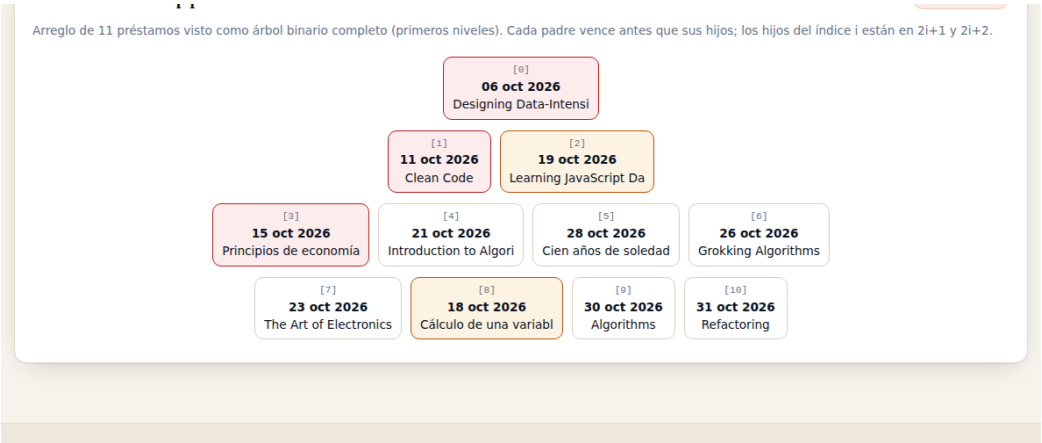


Figura 5. El min-heap visto como árbol binario completo: índices [i], 2i+1 y 2i+2.

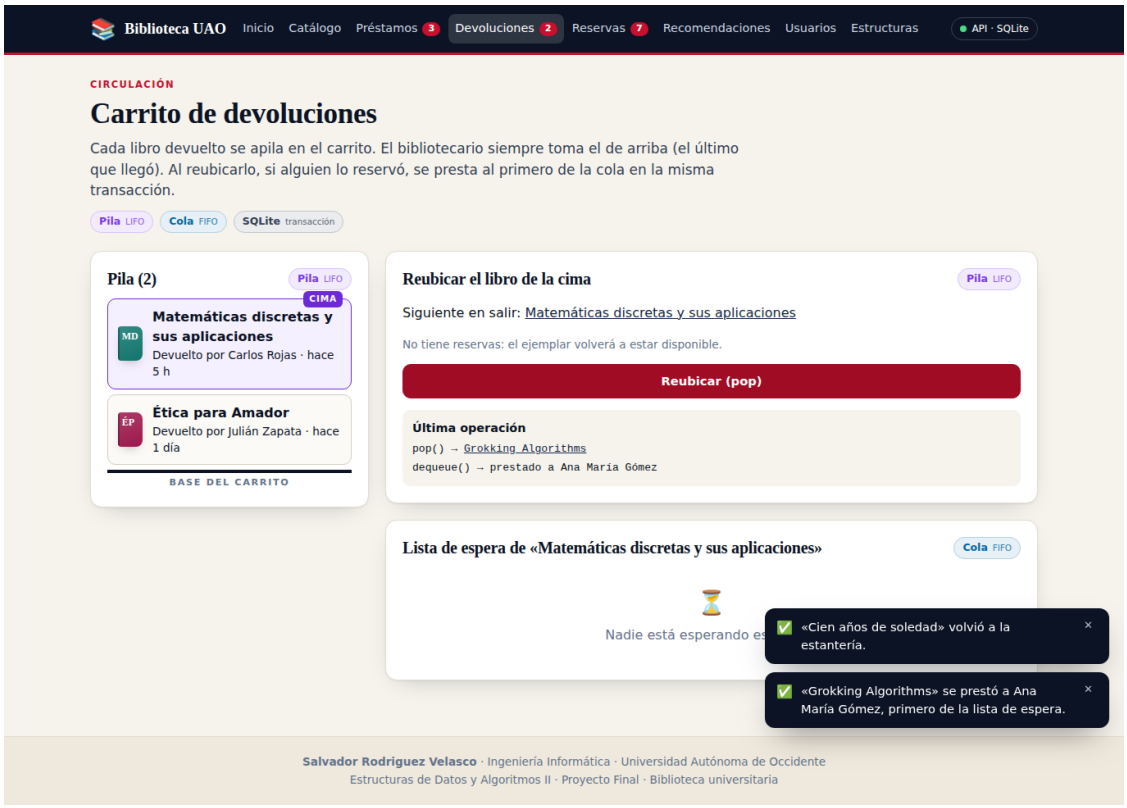


Figura 6. Carrito de devoluciones (Pila): pop() de Grokking Algorithms y dequeue() que se lo presta a Ana María Gómez.

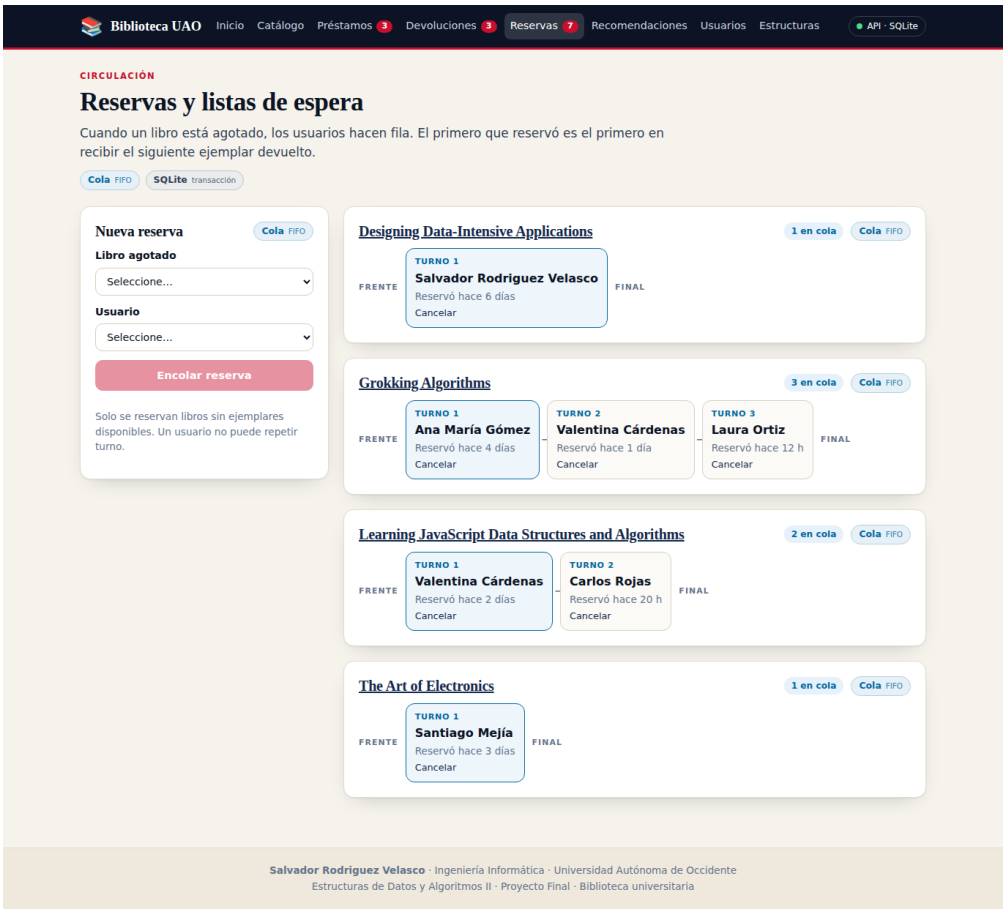


Figura 7. Listas de espera (una Cola por libro agotado).

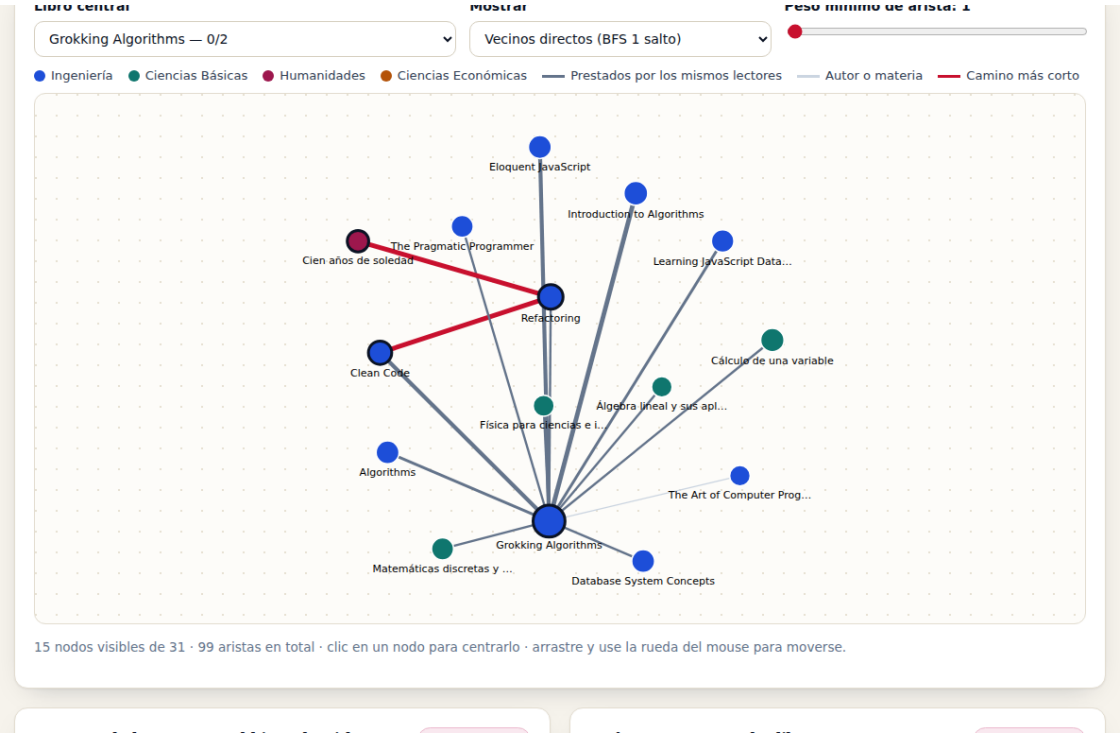


Figura 8. Grafo de recomendaciones con react-d3-graph y camino más corto (BFS) resaltado en rojo.

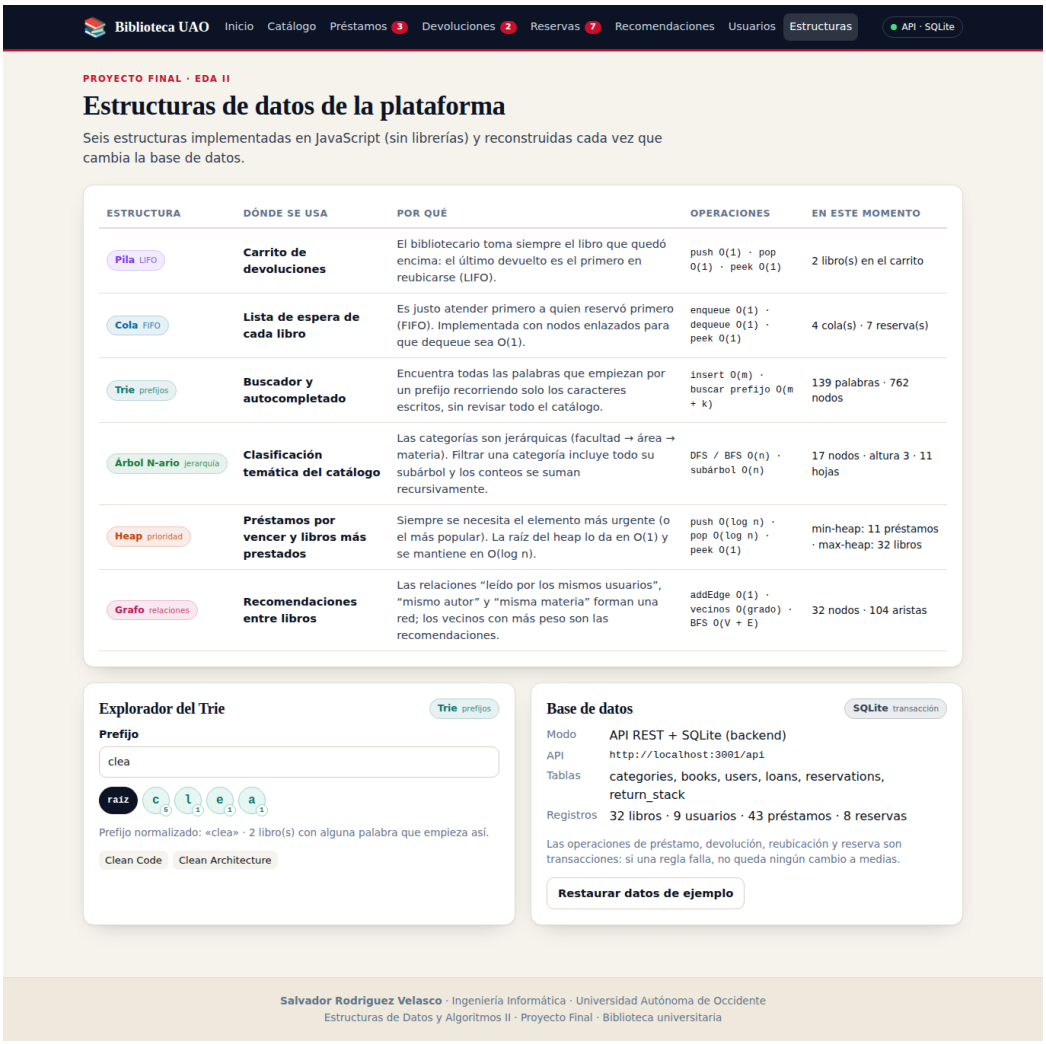


Figura 9. Página de estructuras: uso, justificación y números en vivo; explorador del Trie y estado de la base de datos.

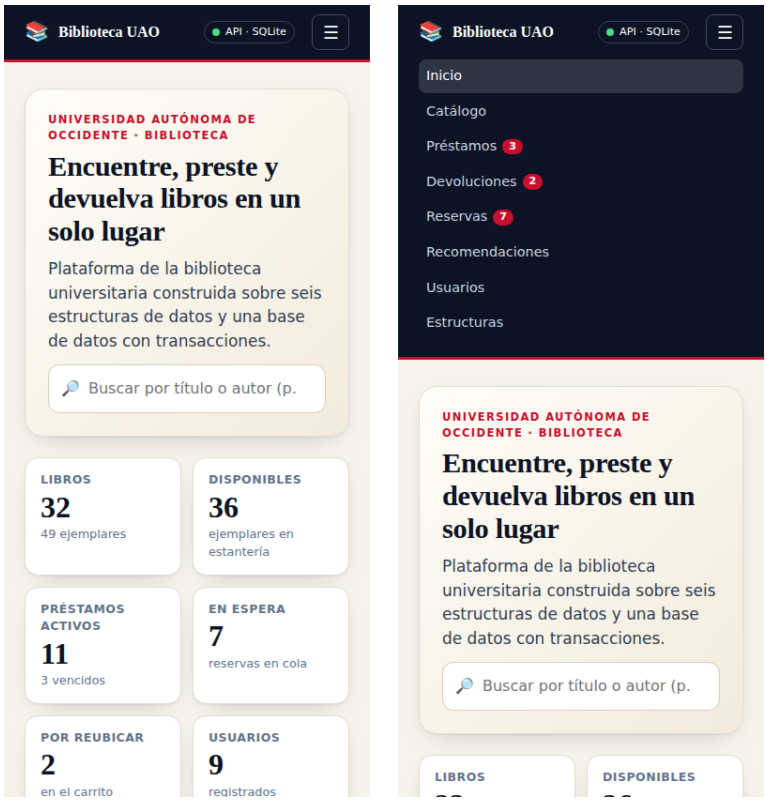


Figura 10. Vista en celular con el menú desplegado.

Paso 8. Pruebas automáticas

El proyecto tiene 17 pruebas con `node:test`. El backend levanta la API con una base en memoria y prueba préstamos, reglas, devoluciones (push), la reubicación con pop y dequeue, las reservas, las validaciones, el ROLLBACK ante una falla simulada y las restricciones CHECK. Las pruebas del frontend verifican cada estructura con los ejemplos de clase, el modelo completo con los datos de ejemplo y las transacciones del modo local.

```
> npm test
ok 1 - health y estado inicial de la base de datos
ok 2 - préstamo: crea el préstamo y descuenta un ejemplar en la misma transacción
ok 3 - préstamo rechazado: sin ejemplares, usuario con vencidos o ya lo tiene
ok 4 - devolución: cierra el préstamo y lo APILA en el carrito (push)
ok 5 - reubicar: POP de la pila + DEQUEUE de la cola en una transacción
ok 6 - reservas: solo sin ejemplares, sin duplicados, y cancelación
ok 7 - libros y usuarios nuevos con validación
ok 8 - transacción: si falla a mitad de camino se hace ROLLBACK
ok 9 - rutas y JSON inválidos
# pass 9   # fail 0
ok 1 - Pila: LIFO (ejemplo de la clase: push 10, push 20, pop, peek)
ok 2 - Cola enlazada: FIFO (ejemplo de la clase: A, B, C)
ok 3 - Trie: inserción, búsqueda exacta, prefijos y normalización
ok 4 - Árbol N-ario: DFS, BFS, altura, subárbol y suma recursiva
ok 5 - Heap: min-heap y max-heap mantienen la raíz y ordenan con pop
ok 6 - Grafo: aristas no dirigidas con peso acumulado, BFS y camino
ok 7 - Modelo de la biblioteca: las 6 estructuras con los datos de ejemplo
ok 8 - Modo local: transacciones atómicas sobre el estado (commit / rollback)
# pass 8   # fail 0
```

Además, se probó la plataforma completa en un navegador automatizado, en modo API y en modo local. Se probaron la búsqueda, el filtro por categoría, el préstamo, la regla de vencidos, la reserva, la devolución, dos reubicaciones, el camino en el grafo, y el alta de usuarios y libros. En celular no hay desbordes horizontales.

Paso 9. Control de versiones con ramas

Cada funcionalidad se desarrolló en su propia rama y se mezcló en `main` con `git merge --no-ff`, para conservar en el historial el aporte de cada rama:

Rama	Contenido
feature/backend-sqlite	API REST, esquema SQLite, transacciones y pruebas
feature/plataforma-base	Repositorio API/local, contexto, diseño y componentes base
feature/pila-cola	Pila (devoluciones) y Cola (reservas)
feature/trie-arbol	Trie (buscador) y Árbol N-ario (categorías)
feature/heap	Heap (vencimientos y ranking)
feature/grafo	Grafo de recomendaciones con react-d3-graph
feature/integracion	Construcción de las estructuras, páginas y pruebas
docs/documentacion	README, documento final, ESTUDIANTE.txt y despliegue

```
git checkout -b feature/pila-cola
git add frontend/src/structures/Stack.js frontend/src/structures/Queue.js ...
git commit -m "feat(pila-cola): ..."
git checkout main
git merge --no-ff feature/pila-cola
```

8. Cómo ejecutar el programa

8.1 Requisitos

- **Node.js 22.13 o superior** (recomendado 22 LTS). Se verifica con `node -v`. Esta versión es necesaria porque el backend usa el módulo nativo `node:sqlite`.
- npm (incluido con Node.js) y un navegador moderno.

8.2 Ejecución completa (backend + frontend)

1. Instalar todo con `npm run install:all`.
2. Terminal 1: iniciar el backend con `npm run dev:backend`. La API queda en http://localhost:3001/api y crea el archivo `backend/data/biblioteca.db` con los datos de ejemplo.

3. Terminal 2: entrar a `frontend`, copiar `.env.example` como `.env` (contiene `VITE_API_URL=http://localhost:3001/api`) y ejecutar `npm run dev`.
4. Abrir http://localhost:5173. La barra superior debe mostrar «API · SQLite».

```
npm run install:all
npm run dev:backend                # terminal 1

cd frontend                        # terminal 2
copy .env.example .env             # Windows  (Linux/Mac: cp .env.example .env)
npm run dev
```

8.3 Solo frontend (modo local)

Sin backend ni archivo `.env`, `npm run dev:frontend` abre la plataforma en «Modo local». Los datos se guardan en el navegador y el botón «Restaurar datos de ejemplo» de la página Estructuras los reinicia.

8.4 Pruebas y compilación

Comando (en la raíz)	Qué hace
`npm test`	Ejecuta las 17 pruebas del backend y de las estructuras.
`npm run build`	Genera la versión de producción del frontend en `frontend/dist`.

8.5 Solución de problemas

Problema	Solución
`No such built-in module: node:sqlite`	La versión de Node.js es menor a 22.13: actualizar a Node.js 22 LTS.
`npm install` del frontend muestra ERESOLVE	Falta `frontend/.npmrc` (`legacy-peer-deps=true`) o usar `npm install --legacy-peer-deps`.
La barra dice «Modo local» con el backend encendido	Revisar que exista `frontend/.env` con `VITE_API_URL` y reiniciar `npm run dev`.
El puerto 3001 o 5173 está ocupado	Backend: `PORT=3002 npm run dev:backend`. Vite usa otro puerto libre automáticamente.
Se quieren los datos iniciales	Página Estructuras › «Restaurar datos de ejemplo» (`POST /api/reset`).

9. Publicación en servidores públicos

9.1 Backend en Render

1. Subir el repositorio a GitHub.

2. En https://render.com: New › Blueprint › seleccionar el repositorio. Render lee `render.yaml` (instala con `npm install --prefix backend` e inicia con `npm start --prefix backend`).
3. Al terminar, abrir `https://<servicio>.onrender.com/api/health`; debe responder `{"ok": true}`.

9.2 Frontend en Netlify

1. En https://app.netlify.com: Add new site › Import an existing project › GitHub › seleccionar el repositorio.
2. Netlify lee `netlify.toml`: base `frontend`, comando `npm run build`, carpeta `dist` y redirección de rutas a `index.html`.
3. En Site configuration › Environment variables agregar `VITE_API_URL = https://<servicio>.onrender.com/api` y volver a desplegar (Deploys › Trigger deploy).
4. Abrir la URL `https://<sitio>.netlify.app`: la barra debe mostrar «API · SQLite».

9.3 Enlaces en el README

Finalmente se reemplazan las URL marcadas con ⚠️ en la tabla «Enlaces» del `README.md` (repositorio, Netlify y Render), se hace commit en una rama y se mezcla en `main`, como pide la rúbrica.

10. Detalle de requisitos

Requisito	Detalle
Plataforma visual con 6 estructuras	React con 6 estructuras: Pila, Cola, Trie, Árbol N-ario, Heap y Grafo (sección 6).
Listas, Pilas o Colas (2)	Pila (carrito de devoluciones) y Cola con nodos enlazados (lista de espera).
Árboles, Tries o Heaps (2)	Trie (buscador), Árbol N-ario (categorías) y Heap (vencimientos y ranking).
Grafos	Grafo ponderado de recomendaciones con BFS y camino más corto, dibujado con react-d3-graph.
Publicación en servidores públicos	`netlify.toml` (frontend) y `render.yaml` (backend) listos; pasos en la sección 9.
Publicación del backend	API Express + SQLite desplegable en Render con health check.
Aporte en ramas	Una rama por funcionalidad, todas mezcladas en `main` con `--no-ff` (paso 9).
Base de datos y transacciones	SQLite con 6 tablas, restricciones CHECK y transacciones BEGIN/COMMIT/ROLLBACK (sección 5).

Requisito	Detalle
Documento final en Git	Este documento, en `docs/` del repositorio.
README con integrantes y enlaces	`README.md` con integrantes, enlaces de despliegue, estructuras, API y ramas.

11. Conclusiones

- Cada estructura se eligió porque modela de forma natural un proceso real de la biblioteca: la pila es el carrito, la cola es la fila de reservas, el Trie es el buscador, el árbol es la clasificación, el heap marca las urgencias y el grafo encuentra relaciones.
- La reubicación de devoluciones combina dos estructuras (pop de la pila y dequeue de la cola) en una sola transacción de base de datos, lo que garantiza que un ejemplar nunca quede “a medias” entre estantería y préstamo.
- Separar las estructuras (`structures/`), su construcción desde la base de datos (`buildLibrary`) y la persistencia (repositorio API o local) permitió probar cada parte de forma independiente con 17 pruebas automáticas.
- Implementar la cola con nodos enlazados y el heap sobre un arreglo muestra cómo la representación interna cambia la complejidad: dequeue O(1) en lugar de O(n), y el más urgente en O(1).
- El modo local con las mismas reglas transaccionales hace que la plataforma publicada funcione, aunque el backend gratuito esté apagado.